Introducción a los grafos

Cuando los datos dejan de acostarse en una línea

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre de 2026

Objetivos de la sesión

Al terminar la clase, usted podrá

- Decir por qué una lista, una pila o un mapa no alcanzan para guardar una red, y qué operación les falta.
- Escribir la definición formal de grafo y distinguir los grafos dirigidos, los no dirigidos y los que llevan peso.
- Calcular grados y usar el teorema del apretón de manos para descartar grafos imposibles.
- Construir las tres representaciones de un grafo en Python a partir de la lista de aristas.
- Escoger la representación que le sirve a un algoritmo, con el costo de cada operación en la mano.

Contenido

1. De las estructuras lineales a los grafos
2. Introducción a los grafos
3. Terminología
4. Familias de grafos simples
5. Topologías y subgrafos
6. Grafos complementarios
7. Representaciones
8. Cuál representación usar
9. Ejercicios
10. Referencias

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

8 Cuál representación usar

9 Ejercicios

10 Referencias

Lo que traemos

Tres nombres que conviene separar

- **Estructura de datos**: determina la forma en que se representa y se organiza la información.
- **Tipo de dato**: la representación particular de un conjunto de valores, con sus valores posibles y sus operaciones.
- **Tipo abstracto de dato**: la definición de un conjunto de valores, sus operaciones y sus propiedades, sin comprometerse con una implementación.

Lo que traemos

La distinción que se va a usar hoy

Una cola es un tipo abstracto: encolar, desencolar, mirar el frente. Que por debajo sea un arreglo circular o una lista enlazada no cambia lo que la cola promete. Un grafo también va a ser un tipo abstracto, y va a tener tres implementaciones distintas.

Las estructuras lineales

Lo que ofrece cada una

Estructura	**Operación**	**Costo**
Arreglo, lista nativa	acceso por índice	$O(1)$
Lista enlazada	insertar o borrar al frente	$O(1)$
Pila	apilar, desapilar	$O(1)$
Cola	encolar, desencolar	$O(1)$
Mapa	consultar por llave	$O(1)$ promedio
Conjunto	pertenencia	$O(1)$ promedio

Las estructuras lineales

Lo que tienen en común

Todas ordenan los datos uno detrás de otro, o los indexan por una llave. Contestan “¿cuál sigue?” y “¿está este valor?”. Ninguna contesta “¿quién está conectado con quién?”.

Direccionamiento directo y conjuntos

Por posición

Índice	Valor
0	a
1	b
2	c
3	d
4	e

Arreglos, vectores, listas nativas.

Por llave

Llave	Valor
k1	a
k2	b
k3	c
k4	d
k5	e

Mapas y diccionarios.

Conjuntos

$\{a, b, c, d, e\}$: los elementos están, sin orden y sin repetición. La pregunta que contestan es la pertenencia.

Datos que no se acuestan en una línea

Jerarquías

Laborales, sociales, económicas, políticas: cada elemento tiene encima y debajo a otros, y no siempre uno solo.

Conexiones

Redes sociales, redes eléctricas, redes viales: lo que importa no es el orden de los elementos sino qué par de elementos está unido.

Las dos preguntas

- ¿Es posible representar estos datos con las estructuras anteriores?
- ¿Hay algún tipo abstracto de dato debajo de esta información?

El intento con lo que ya tenemos

Amistades entre cinco personas

Ana y Beto son amigos; Ana y Carlos también; Beto y Diana; Elena no conoce a nadie.

Primer intento: una lista

```
personas = ["Ana", "Beto", "Carlos", "Diana", "Elena"]
```

La lista guarda quiénes son, y ahí se detiene. La amistad es una relación entre dos, y en la lista no cabe.

El intento con lo que ya tenemos

Segundo intento: un mapa de listas

```
amigos = {"Ana": ["Beto", "Carlos"], "Beto": ["Ana", "Diana"],
          "Carlos": ["Ana"], "Diana": ["Beto"], "Elena": []}
```

Esto sí guarda la relación, y está hecho con las estructuras de siempre. Lo que falta no es la implementación: falta nombrar la estructura, definir sus operaciones y saber qué cuesta cada una.

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

8 Cuál representación usar

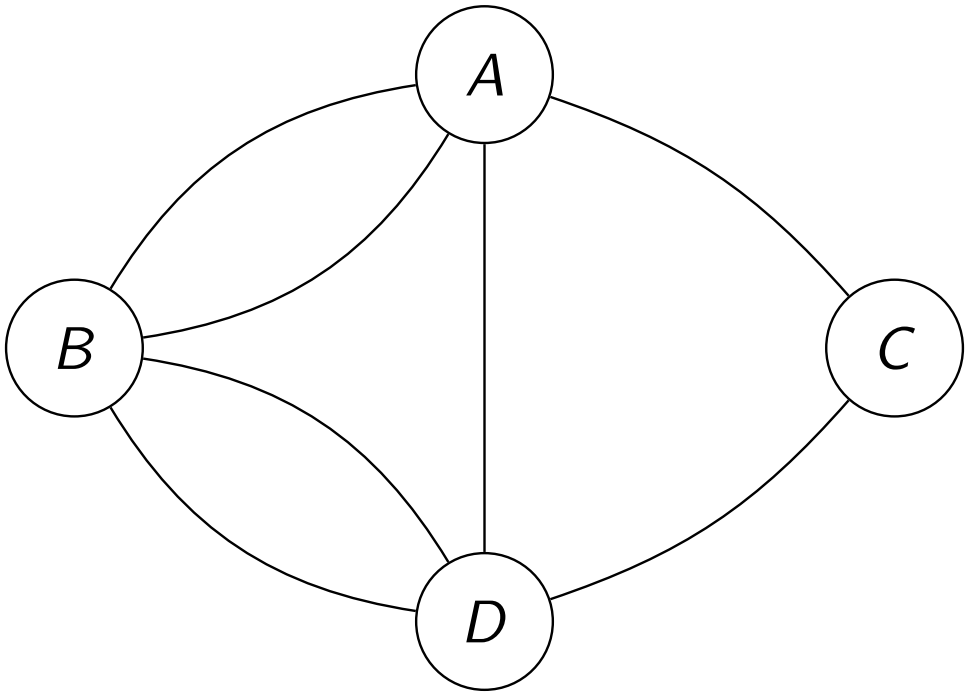
9 Ejercicios

10 Referencias

Los puentes de Königsberg

El problema de 1736

La ciudad estaba partida por el río Pregel, que formaba dos islas unidas entre sí y con las orillas por siete puentes. La pregunta de sus habitantes: ¿se puede salir de cualquier punto, cruzar cada puente exactamente una vez y volver al punto de partida?



Los puentes de Königsberg

Lo que hizo Euler

Demostró que ese recorrido no existe. Su argumento no usa distancias ni formas: solo cuántos puentes llegan a cada región. Ese descarte de todo lo demás es lo que queda cuando uno se queda con el grafo.

Definición de grafo

Definición (Grafo, CLRS Apéndice B.4, p. 1168)

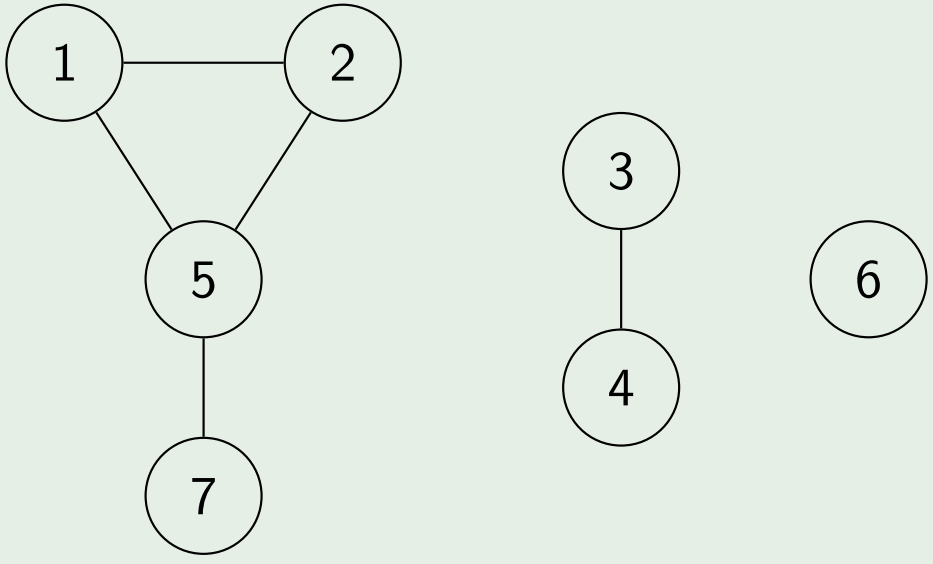
Un grafo G es una pareja (V, E) donde

- $V = \{v_1, v_2, \ldots, v_n\}$ *es un conjunto de nodos o vértices, y*
- $E = \{(a_1, b_1), (a_2, b_2), \ldots, (a_m, b_m)\}$ *es un conjunto de arcos o aristas entre elementos de V, es decir, $a_i \in V$ y $b_i \in V$ para todo i.*

Definición de grafo

Un grafo de siete vértices y cinco aristas

$V = \{1, 2, 3, 4, 5, 6, 7\}$ y $E = \{\{1, 2\}, \{1, 5\}, \{2, 5\}, \{3, 4\}, \{5, 7\}\}$.



El vértice 6 no tiene aristas y sigue siendo parte del grafo.

Propiedades básicas

Grafo vacío

El grafo $(\emptyset, \emptyset)$ se escribe $\emptyset$: no tiene vértices ni aristas.

Orden

El número de vértices de G es su orden, denotado $|G|$.

Grafo trivial

Un grafo de orden 0 o 1. Un grafo puede ser finito o infinito según su orden; en el curso son siempre finitos.

Un grafo sin ninguna arista

$V = \{a, b, c, d, e, f, g, h\}$ y $E = \emptyset$: ocho vértices sueltos. Sigue siendo un grafo, y más adelante va a ser el caso límite de varios algoritmos.

Grafo simple

Definición (Grafo simple)

Un grafo simple $G = (V, E)$ cumple tres condiciones:

- *cada arista conecta dos vértices diferentes;*
- *no hay aristas paralelas, es decir, dos aristas nunca conectan el mismo par de vértices;*
- *no hay bucles, aristas que unan un vértice consigo mismo.*

Por qué es el caso de referencia

Al no permitir aristas múltiples ni bucles, la relación entre dos vértices es binaria: están conectados o no lo están. Los demás tipos se definen relajando alguna de las tres condiciones.

Multigrafo y pseudografo

Definición (Multigrafo)

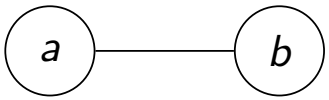
Un multigrafo $G = (V, E)$ consta de un conjunto V de vértices, un conjunto E de aristas y una función f de E en $\{\{u, v\} \mid u, v \in V,\ u \neq v\}$. Las aristas e_1 y e_2 son paralelas si $f(e_1) = f(e_2)$. Permite aristas múltiples, no bucles.

Definición (Pseudografo)

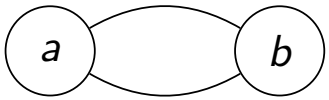
Un pseudografo $G = (V, E)$ tiene la misma forma con f de E en $\{\{u, v\} \mid u, v \in V\}$. Una arista e es un bucle si $f(e) = \{u, u\} = \{u\}$. Permite aristas paralelas y bucles.

Multigrafo y pseudografo

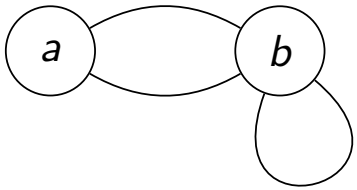
simple



multigrafo



pseudografo



Grafos dirigidos y no dirigidos

Definición (No dirigido)

$G = (V, E)$ con V conjunto de vértices y $E \subseteq V \times V$ tal que E es simétrica: si $(u, v) \in E$ entonces $(v, u) \in E$.

Definición (Dirigido)

$G = (V, E)$ con V conjunto de vértices y $E \subseteq V \times V$. Cada $(a, b) \in E$ es un arco que va de a hacia b.

La diferencia que importa

En el dirigido, (u, v) no es lo mismo que (v, u). Que exista el arco de u a v no dice nada sobre el arco de v a u. Un grafo no dirigido es el caso particular en que los dos siempre van juntos.

Grafos dirigidos y no dirigidos

Multigrafo dirigido

Igual al multigrafo, con f de E en $\{(u, v) \mid u, v \in V\}$: los arcos tienen dirección y puede haber varios entre el mismo par.

Los cinco tipos

Tipo	**Aristas**	**Paralelas**	**Bucles**
Grafo simple	no dirigidas	no	no
Multigrafo	no dirigidas	sí	no
Pseudografo	no dirigidas	sí	sí
Grafo dirigido	dirigidas	no	sí
Multigrafo dirigido	dirigidas	sí	sí

Cómo se clasifica uno que le den

1. Mirar si las aristas tienen dirección.
2. Buscar dos aristas entre el mismo par de vértices.
3. Buscar aristas de un vértice a sí mismo.

Los cinco tipos

Lo que se usa en el curso

Casi todo lo que viene trabaja sobre grafos simples y sobre grafos dirigidos. Los otros tres aparecen cuando el problema los impone, como los siete puentes.

Grafos con peso

Definición (Grafo con peso)

Un grafo dirigido con peso es un grafo $G = (V, E)$ *con una función de peso* $f : E \to \mathbb{R}$.

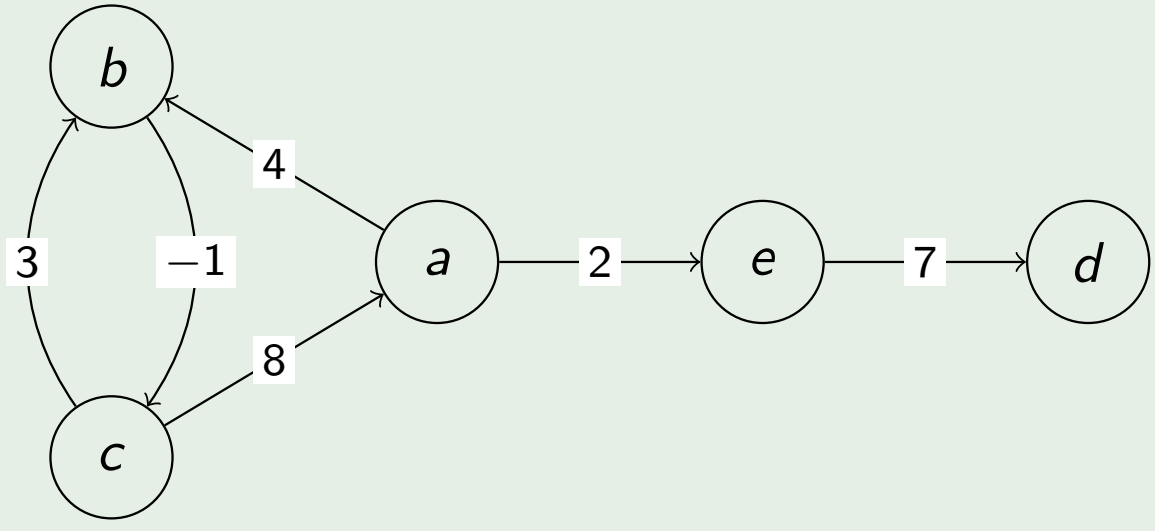
El peso no tiene por qué ser una distancia

Puede ser tiempo, costo, capacidad o probabilidad. Y puede ser negativo: eso va a decidir cuál algoritmo de caminos más cortos sirve.

Grafos con peso

Un grafo con peso

$V = \{a, b, c, d, e\}$, $E = \{(a, b), (a, e), (b, c), (c, a), (c, b), (e, d)\}$, con $f((a, b)) = 4$, $f((a, e)) = 2$, $f((b, c)) = -1$, $f((c, a)) = 8$, $f((c, b)) = 3$, $f((e, d)) = 7$.



Dónde aparecen

Amigos en una red social

Cada persona es un vértice y cada amistad una arista. Sin dirección si la amistad es mutua, con dirección si es un "sigue a".

Redes

Los nodos son equipos o postes y las aristas los enlaces. Aquí el peso suele ser distancia, latencia o capacidad.

Rutas del MIO

Las estaciones son vértices y los tramos de ruta son aristas. Universidades, Univalle, Buitrera, Capri, Pampalinda, Plaza de Toros, Unidad Deportiva por un lado; Calipso y Menga por otro.

Árbol genealógico

Las personas son vértices y las aristas van de padre a hijo. Un árbol es un grafo con una restricción encima, y esa restricción se estudia al final del curso.

Dónde aparecen

Lo que se gana al modelar así

Un algoritmo escrito sobre grafos sirve para los cuatro casos. La ruta más corta entre dos estaciones del MIO y el grado de separación entre dos personas son el mismo problema.

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

8 Cuál representación usar

9 Ejercicios

10 Referencias

Adyacencia, incidencia y vecindad

Definición (Adyacencia)

Dos vértices u y v de un grafo no dirigido G son adyacentes, o vecinos, si $\{u, v\}$ es una arista de G.

Definición (Incidencia)

Si $e = \{u, v\}$, la arista e es incidente con los vértices u y v, y conecta a u con v. Los vértices u y v son los extremos de e. Dos aristas son adyacentes si comparten un vértice.

Definición (Vecindad)

La vecindad de un vértice x, denotada $N(x)$, es el conjunto de todos los vértices adyacentes a x.

Adyacencia, incidencia y vecindad

En el grafo de siete vértices

$N(5) = \{1, 2, 7\}$, $N(3) = \{4\}$ y $N(6) = \emptyset$.

Grado de un vértice

Definición (Grado)

El grado de un vértice v en un grafo no dirigido es el número de aristas incidentes con él. Se denota $\delta(v)$. Los bucles cuentan dos veces, porque tocan al vértice por sus dos extremos.

Dos nombres

Un vértice de grado 0 es **aislado**. Uno de grado 1 es **colgante**.

Grados del grafo de siete vértices

v	1	2	3	4	5	6	7
$\delta(v)$	2	2	1	1	3	0	1

El 6 es aislado; el 3, el 4 y el 7 son colgantes.

Teorema del apretón de manos

Teorema (Apretón de manos, CLRS Ejercicio B.4-1)

Sea $G = (V, E)$ *un grafo no dirigido con* e *aristas. Entonces*

$$2e = \sum_{v \in V} \delta(v).$$

Teorema del apretón de manos

Demostración

Se procede de forma directa, contando el mismo objeto de dos maneras. Un par (arista, extremo suyo) se puede contar recorriendo las aristas, y cada arista aporta exactamente 2 de esos pares, uno por cada extremo; el total es $2e$. El mismo par se puede contar recorriendo los vértices, y cada vértice v aporta $\delta(v)$ pares, uno por cada arista incidente con él; el total es $\sum_{v \in V} \delta(v)$. Como los dos conteos cuentan lo mismo, las dos cantidades son iguales. Por lo tanto, se puede concluir que $2e = \sum_{v \in V} \delta(v)$.

En el grafo de siete vértices

$\sum_v \delta(v) = 2 + 2 + 1 + 1 + 3 + 0 + 1 = 10 = 2 \cdot 5$, y en efecto tiene 5 aristas.

Para qué sirve contar así

Contar aristas sin dibujar el grafo

¿Cuántas aristas tiene un grafo de 10 vértices, todos de grado 6?
$2e = 10 \cdot 6 = 60$, luego $e = 30$.

Descartar un grafo imposible

¿Existe un grafo simple de 15 vértices, todos de grado 5?
$2e = 15 \cdot 5 = 75$, que es impar. No existe, y no hubo que intentar dibujarlo.

Corolario

Todo grafo no dirigido tiene un número par de vértices de grado impar.

Para qué sirve contar así

Demostración

Se procede de forma directa. Sea V_1 el conjunto de vértices de grado par y V_2 el de grado impar. Entonces

$$2e = \sum_{v \in V} \delta(v) = \sum_{v \in V_1} \delta(v) + \sum_{v \in V_2} \delta(v).$$

El lado izquierdo es par por ser $2e$, y $\sum_{v \in V_1} \delta(v)$ es par por ser suma de números pares. Al despejar, $\sum_{v \in V_2} \delta(v)$ es diferencia de dos pares, luego es par. Cada sumando de esa suma es impar, y una suma de impares es par únicamente cuando hay un número par de sumandos. Por lo tanto, se puede concluir que $|V_2|$ es par.

Grados en grafos dirigidos

Definición (Grado de entrada y de salida, CLRS Apéndice B.4)

En un grafo dirigido, el grado de entrada $\delta^-(v)$ es el número de arcos que tienen a v como vértice final, y el grado de salida $\delta^+(v)$ es el número de arcos que tienen a v como vértice inicial. Si (u, v) es un arco, u es el inicial y v el final.

Teorema

Sea $G = (V, E)$ un grafo dirigido. Entonces

$$\sum_{v \in V} \delta^-(v) = \sum_{v \in V} \delta^+(v) = |E|.$$

Grados en grafos dirigidos

Demostración

Se procede de forma directa. Cada arco (u, v) tiene exactamente un vértice inicial y exactamente uno final, de modo que aporta 1 a $\delta^+(u)$ y 1 a $\delta^-(v)$, y no aporta nada a los demás vértices. Al sumar sobre todos los vértices, cada una de las dos sumas cuenta cada arco una sola vez. Por lo tanto, se puede concluir que las dos sumas valen $|E|$.

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

8 Cuál representación usar

9 Ejercicios

10 Referencias

Grafo completo K_n

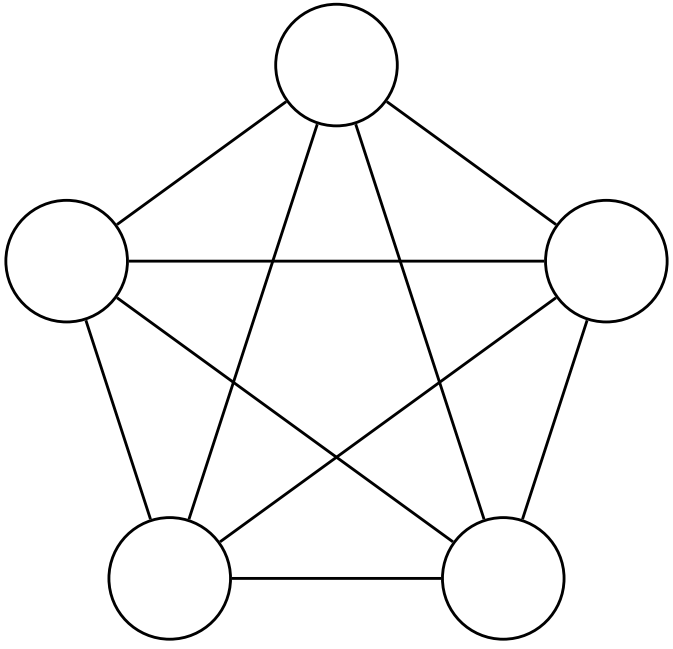
Definición (Grafo completo)

El grafo completo de n vértices, denotado K_n, es el grafo simple que tiene exactamente una arista entre cada par de vértices distintos.

Cuántas aristas tiene

Cada vértice tiene grado $n-1$, así que por el apretón de manos $2e = n(n-1)$ y entonces $e = \dfrac{n(n-1)}{2}$. Para $n = 1, \ldots, 6$ eso da $0, 1, 3, 6, 10, 15$.

Grafo completo K_n



K_5

El límite superior de todo

K_n es el grafo con más aristas posible sobre n vértices. Cuando se dice que un algoritmo cuesta $O(V^2)$ en el peor caso, el peor caso es este.

Ciclos C_n y ruedas W_n

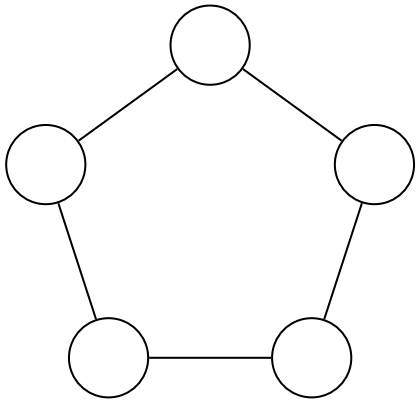
Definición (Ciclo)

El ciclo C_n, con $n \geq 3$, tiene vértices $v_1, \ldots, v_n$ y aristas $\{v_1, v_2\}, \{v_2, v_3\}, \ldots, \{v_{n-1}, v_n\}, \{v_n, v_1\}$. Cada vértice tiene grado 2 y hay n aristas.

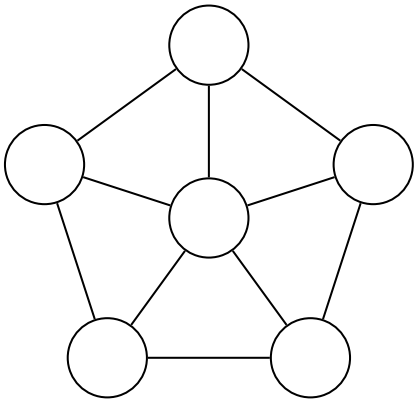
Definición (Rueda)

La rueda W_n se obtiene agregando al ciclo C_n un vértice nuevo y conectándolo con los n vértices del ciclo. Tiene $n+1$ vértices y $2n$ aristas; el centro tiene grado n y los demás grado 3.

Ciclos C_n y ruedas W_n



C_5



W_5

Bipartito completo y conteo por familia

Definición (Bipartito completo)

$K_{m,n}$ tiene sus vértices partidos en dos grupos, de tamaños m y n, sin aristas dentro de cada grupo y con todas las aristas posibles entre los dos.

Familia	**Vértices**	**Aristas**	**Grado**
K_n	n	$n(n-1)/2$	$n-1$
C_n	n	n	2
W_n	$n+1$	$2n$	no regular
$K_{m,n}$	$m+n$	$m \cdot n$	regular solo si $m = n$

Definición (Secuencia de grado)

La lista de los grados de los vértices de G, ordenada de forma decreciente. La de K_4 es $3,3,3,3$; la de C_4 es $2,2,2,2$; la de W_4 es $4,3,3,3,3$; la de $K_{2,3}$ es $3,3,2,2,2$.

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

8 Cuál representación usar

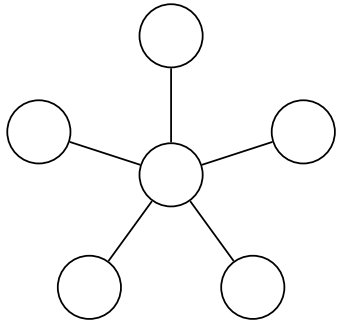
9 Ejercicios

10 Referencias

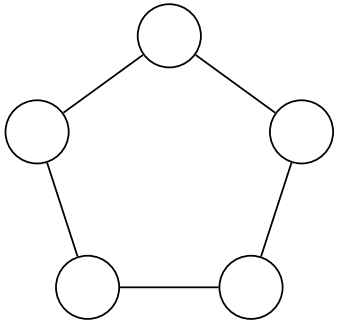
Topologías de red

Tres formas conocidas

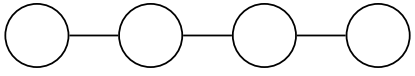
- **Estrella**: un nodo central conectado con todos los demás. Es $K_{1,n}$. Si el central falla, la red entera cae.
- **Anillo**: cada nodo conectado con exactamente dos vecinos. Es C_n, y sobrevive a la caída de un enlace.
- **Lineal**: los nodos en fila, el camino P_n. Aparece en arreglos de procesadores en cómputo paralelo.



estrella



anillo



lineal

Subgrafos

Definición (Subgrafo, CLRS Apéndice B.4)

Si $G = (V, E)$ es un grafo, entonces $G_1 = (V_1, E_1)$ es un subgrafo de G si $V_1 \subseteq V$ y $E_1 \subseteq E$, con la condición de que las aristas de E_1 conecten vértices que estén en V_1.

Recubridor

G_1 es recubridor si $V_1 = V$: conserva todos los vértices y puede perder aristas.

Inducido

Para $W \subseteq V$, el subgrafo inducido $\langle W \rangle$ tiene los vértices de W y **todas** las aristas de G con sus dos extremos en W. No se puede omitir ninguna.

Subgrafos

Eliminar

Al quitar una arista e queda $G - \{e\}$, con los mismos vértices. Al quitar un vértice v queda $G - \{v\}$, que se lleva también todas las aristas incidentes con v. Estas dos operaciones son la base del análisis de conectividad y de la búsqueda de puntos de articulación.

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

8 Cuál representación usar

9 Ejercicios

10 Referencias

Complemento

Definición (Complemento)

Sea G un grafo simple no dirigido de n vértices. Su complemento $\overline{G}$ tiene los mismos vértices, y dos de ellos son adyacentes en $\overline{G}$ si y solo si no lo son en G.

Cuántas aristas

Si G tiene n vértices y e aristas, entonces $\overline{G}$ tiene $\frac{n(n-1)}{2} - e$ aristas: las que le faltaban a G para ser completo.

Casos extremos

El complemento de K_n es el grafo de n vértices sin ninguna arista, llamado grafo nulo. El complemento del grafo nulo es K_n.

Complemento

Secuencia de grado

Si la de G es $d_1, d_2, \ldots, d_n$, la de $\overline{G}$ es $n-1-d_n, n-1-d_{n-1}, \ldots, n-1-d_1$.

Unión de grafos

Definición (Unión)

La unión de dos grafos simples $G_1 = (V_1, E_1)$ y $G_2 = (V_2, E_2)$ es el grafo simple con vértices $V_1 \cup V_2$ y aristas $E_1 \cup E_2$, denotado $G_1 \cup G_2$.

Teorema

Si G es un grafo simple de n vértices, entonces $G \cup \overline{G} = K_n$.

Demostración

Se procede de forma directa. Tómese un par cualquiera de vértices distintos u y v. Por la definición de complemento, $\{u, v\}$ es arista de G o es arista de $\overline{G}$, y nunca deja de estar en las dos. Entonces $\{u, v\}$ pertenece a $E \cup \overline{E}$, que es el conjunto de aristas de la unión. Como el par era cualquiera, la unión tiene una arista entre cada par de vértices distintos. Por lo tanto, se puede concluir que $G \cup \overline{G} = K_n$.

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

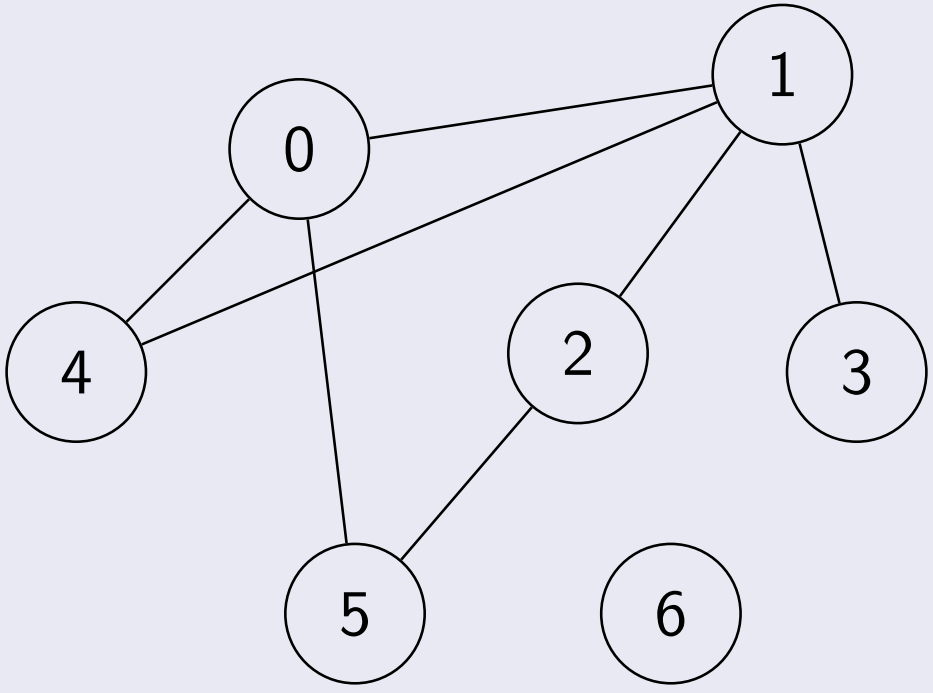
8 Cuál representación usar

9 Ejercicios

10 Referencias

El grafo con el que se va a trabajar

G_1: siete vértices, siete aristas, no dirigido



Aristas: $\{0,1\}, \{0,4\}, \{0,5\}, \{1,2\}, \{1,3\}, \{1,4\}, \{2,5\}$. El vértice 6 queda aislado.

El grafo con el que se va a trabajar

Los vértices se numeran desde cero

Los enunciados suelen numerar de 1 a n y Python indexa desde 0. Se corren los índices al leer, una sola vez, y de ahí en adelante todo el programa trabaja en $0..n-1$. Mezclar las dos numeraciones es el error más frecuente al empezar con grafos.

Lista de adyacencia

Definición (Lista de adyacencia, CLRS Sección 22.1, p. 589)

El grafo se representa como una lista G de listas, donde $G[u]$ contiene los vértices adyacentes a u.

G_1 en lista de adyacencia

```
G1 = [[4, 5, 1],      # 0
      [0, 2, 3, 4],   # 1
      [1, 5],         # 2
      [1],            # 3
      [0, 1],         # 4
      [0, 2],         # 5
      []]             # 6
```

Lista de adyacencia

Cuánto ocupa

Una entrada por vértice, y dentro de ellas una entrada por cada extremo de cada arista. En el grafo no dirigido cada arista aparece dos veces, de modo que el espacio es $\Theta(V + E)$.

Construir la lista de adyacencia

```python
def lista_de_adyacencia(n, aristas, dirigido):
    # G[u] guarda los vecinos de u: una lista por vertice
    G = []
    u = 0
    while u < n:
        G.append([])
        u = u + 1
    for arista in aristas:
        u = arista[0]
        v = arista[1]
        G[u].append(v)
        if not dirigido:
            G[v].append(u)
    return G
```

Lo único que cambia entre dirigido y no dirigido

La línea que agrega el par al revés. Sin ella, la arista solo se puede recorrer de u hacia v.

Cuando los vértices no son números

El problema

Los datos llegan con nombres: personas, estaciones, ciudades. Las listas y las matrices se indexan con enteros.

Un mapa de nombre a índice

```
id = {"pepito": 0, "luisito": 1, "martita": 2,
      "juanito": 3, "maria": 4, "sofia": 5}

G = [[2, 3],      # pepito
     [0, 2, 4],   # luisito
     [0, 4, 5],   # martita
     [4, 5],      # juanito
     [1],         # maria
     [3]]         # sofia
```

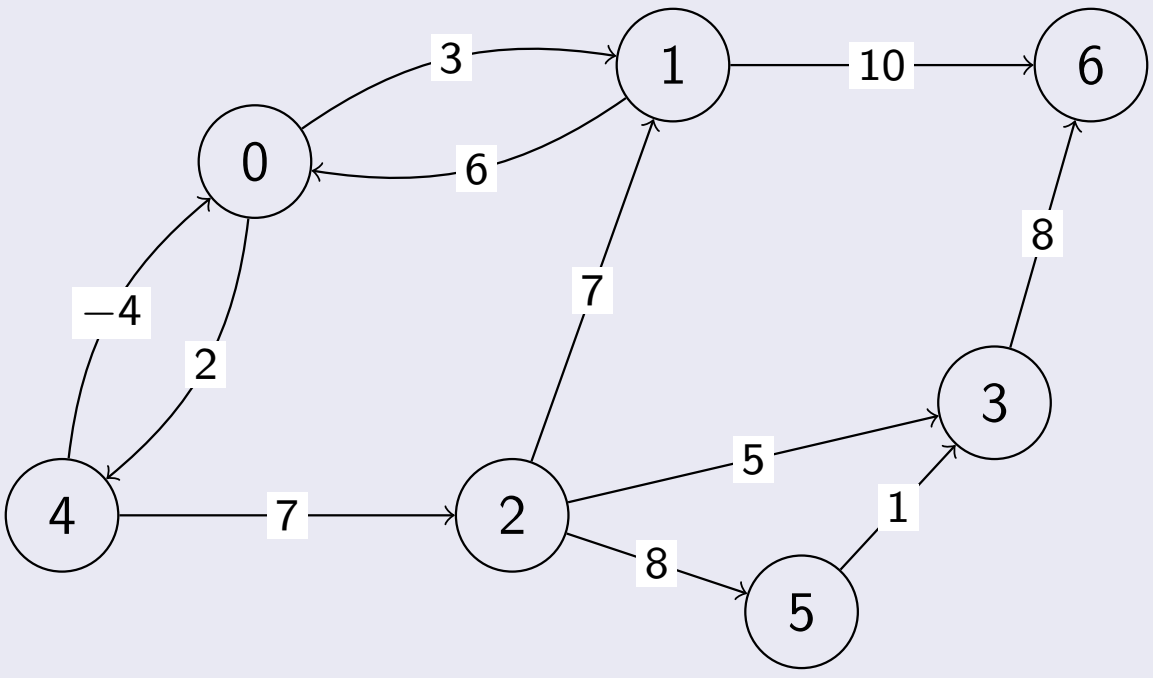
Cuando los vértices no son números

La alternativa directa

También se puede usar un diccionario de listas, `{"pepito": ["martita", "juanito"], ...}`. Se lee mejor y se paga con una búsqueda por llave en cada acceso; el mapa a enteros deja el resto del programa trabajando con índices.

Lista de adyacencia con pesos

El grafo G_4, dirigido y con peso



Lista de adyacencia con pesos

Dos formas de guardarlo

- Dos listas en paralelo: G con los vecinos y w con los pesos, de modo que `w[u][i]` es el peso de la arista que va a `G[u][i]`.
- Una sola lista de parejas: $G[u]$ guarda `(vecino, peso)`.

Matriz de adyacencia

Definición (Matriz de adyacencia, CLRS Sección 22.1, p. 590)

El grafo se representa como una matriz m de $n \times n$ donde m_{ij} vale 1 si hay una arista entre el vértice i y el vértice j, y 0 en caso contrario.

G_1 en matriz de adyacencia

```
G1 = [[0, 1, 0, 0, 1, 1, 0],   # 0
      [1, 0, 1, 1, 1, 0, 0],   # 1
      [0, 1, 0, 0, 0, 1, 0],   # 2
      [0, 1, 0, 0, 0, 0, 0],   # 3
      [1, 1, 0, 0, 0, 0, 0],   # 4
      [1, 0, 1, 0, 0, 0, 0],   # 5
      [0, 0, 0, 0, 0, 0, 0]]   # 6
```

Matriz de adyacencia

Lo que se ve en la matriz y no en la lista

En un grafo no dirigido la matriz es simétrica, $m_{ij} = m_{ji}$. La diagonal es cero en un grafo simple, y un 1 en la diagonal es un bucle. El espacio es $\Theta(V^2)$ aunque el grafo casi no tenga aristas: la fila del vértice 6 ocupa lo mismo que las demás.

Construir la matriz de adyacencia

```
def matriz_de_adyacencia(n, aristas, dirigido):
    # m[u][v] vale 1 si la arista existe y 0 si no existe
    m = []
    u = 0
    while u < n:
        fila = []
        v = 0
        while v < n:
            fila.append(0)
            v = v + 1
        m.append(fila)
        u = u + 1
    for arista in aristas:
        u = arista[0]
        v = arista[1]
        m[u][v] = 1
        if not dirigido:
            m[v][u] = 1
    return m
```

Construir la matriz de adyacencia

Construir la matriz ya cuesta $\Theta(V^2)$

Llenarla de ceros toma n^2 pasos antes de mirar una sola arista. En un grafo de 10^5 vértices eso no cabe en memoria, y ahí la representación queda descartada sin discusión.

Matriz de adyacencia con pesos

Qué se guarda en cada posición

El peso de la arista si existe, e ∞ si no existe. El 0 deja de servir como marca de ausencia, porque un peso puede valer 0.

G_4 en matriz de adyacencia

```
INF = float("inf")
G4 = [[INF,   3, INF, INF,   2, INF, INF],   # 0
      [  6, INF, INF, INF, INF, INF,  10],   # 1
      [INF,   7, INF,   5, INF,   8, INF],   # 2
      [INF, INF, INF, INF, INF, INF,   8],   # 3
      [ -4, INF,   7, INF, INF, INF, INF],   # 4
      [INF, INF, INF,   1, INF, INF, INF],   # 5
      [INF, INF, INF, INF, INF, INF, INF]]   # 6
```

Matriz de adyacencia con pesos

Dónde reaparece esta forma

Es la matriz con la que arranca Floyd-Warshall, que trabaja sobre la matriz completa y no sobre listas.

Lista de aristas

Definición (Lista de aristas)

El grafo se representa como una lista G cuyos elementos son las aristas del grafo. En un grafo con peso, cada elemento es una tripla $(u, v, peso)$.

G_1 y G_4 en lista de aristas

```
G1 = [(0, 1), (1, 0), (0, 4), (4, 0), (0, 5), (5, 0), (1, 2),
      (2, 1), (1, 3), (3, 1), (2, 5), (5, 2), (1, 4), (4, 1)]

G4 = [(0, 1, 3), (0, 4, 2), (1, 0, 6), (1, 6, 10), (2, 1, 7),
      (2, 3, 5), (2, 5, 8), (3, 6, 8), (4, 2, 7), (4, 0, -4),
      (5, 3, 1)]
```

Cuánto ocupa

$\Theta(E)$, y nada más. Es la representación más compacta y la más parecida a como llega la entrada de un problema.

Matriz de incidencia

Definición (Matriz de incidencia)

Una matriz M de $|V| \times |E|$ donde M_{ij} vale 1 si la arista j es incidente con el vértice i, y 0 en caso contrario.

Dos cosas que se leen de una vez

La suma de la fila i es el grado del vértice i. La columna de una arista tiene exactamente dos unos, uno por cada extremo. En grafos dirigidos se usa 1 para el origen y -1 para el destino.

Casi nunca se usa para programar

Ocupa $\Theta(V \cdot E)$ y no contesta rápido ninguna de las dos preguntas que hacen los algoritmos. Aparece en problemas donde la matriz misma es el dato de entrada.

¿Existe la arista (u, v)?

Lista de adyacencia

```python
def hay_arista_en_lista(G, u, v):
    # Recorre los vecinos de u buscando a v
    encontrada = False
    for w in G[u]:
        if w == v:
            encontrada = True
    return encontrada
```

Matriz de adyacencia

```python
def hay_arista_en_matriz(m, u, v):
    # Una sola consulta: la posicion ya dice la respuesta
    return m[u][v] == 1
```

Lista de aristas

```python
def hay_arista_en_lista_de_aristas(E, u, v):
    # Recorre todas las aristas del grafo, no solo las de u
    encontrada = False
    for arista in E:
        if arista[0] == u and arista[1] == v:
            encontrada = True
    return encontrada
```

¿Existe la arista (u, v)?

Lo que cuesta cada una

La matriz contesta en $O(1)$. La lista de adyacencia recorre los vecinos de u, que en el peor caso son $V - 1$. La lista de aristas recorre las E aristas del grafo.

¿Cuáles son los vecinos de u?

Lista de adyacencia

```python
def vecinos_en_lista(G, u):
    # Los vecinos ya estan juntos: la lista de u es la respuesta
    return G[u]
```

Matriz de adyacencia

```python
def vecinos_en_matriz(m, u):
    # Hay que revisar la fila completa, incluidos los ceros
    resultado = []
    v = 0
    while v < len(m):
        if m[u][v] == 1:
            resultado.append(v)
        v = v + 1
    return resultado
```

Lista de aristas

```python
def vecinos_en_lista_de_aristas(E, u):
    # Hay que revisar todas las aristas del grafo
    resultado = []
    for arista in E:
        if arista[0] == u:
            resultado.append(arista[1])
    return resultado
```

¿Cuáles son los vecinos de u?

Esta es la pregunta que hacen los recorridos

Un recorrido pregunta por los vecinos de cada vértice, una vez por vértice. Sobre listas, la suma de todas esas consultas es $\Theta(V + E)$; sobre la matriz es $\Theta(V^2)$ aunque el grafo tenga tres aristas.

Las tres, lado a lado

	Lista de ady.	**Matriz de ady.**	**Lista de aristas**
Espacio	$\Theta(V + E)$	$\Theta(V^2)$	$\Theta(E)$
¿Existe (u, v)?	$O(\delta(u))$	$O(1)$	$O(E)$
Vecinos de u	$\Theta(\delta(u))$	$\Theta(V)$	$\Theta(E)$
Recorrer todo	$\Theta(V + E)$	$\Theta(V^2)$	$\Theta(E)$
Agregar arista	$O(1)$	$O(1)$	$O(1)$

Comprobado, no supuesto

Las tres construcciones y las dos consultas se verificaron sobre los 64 grafos no dirigidos y los 4096 grafos dirigidos que existen sobre cuatro vértices: en todos contestaron lo mismo.

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

8 Cuál representación usar

9 Ejercicios

10 Referencias

Primero la densidad

Dos extremos

Un grafo es **denso** cuando E se acerca a V^2, y **ralo** cuando E es del orden de V. Las redes viales, las redes sociales y casi todo lo que llega de un problema real son ralos.

Con números

Un grafo de $V = 10^5$ y $E = 2 \cdot 10^5$: la lista de adyacencia guarda $4 \cdot 10^5$ entradas; la matriz guarda 10^{10} posiciones, de las cuales 99,9998 % son ceros.

La regla corta

Si el grafo es ralo, lista de adyacencia. La matriz se justifica cuando V es pequeño, cuando el grafo es denso, o cuando el algoritmo hace muchas consultas de adyacencia sueltas.

Después, qué pregunta hace el algoritmo

Cada algoritmo pide una operación

- Los recorridos en profundidad y en amplitud piden los vecinos de cada vértice, una vez por vértice: **lista de adyacencia**, y con ella cuestan $\Theta(V + E)$.
- Los caminos más cortos entre todos los pares recorren la matriz completa: **matriz de adyacencia**, con ∞ en las aristas que no existen.
- Los algoritmos que relajan o que ordenan todas las aristas recorren E de principio a fin: **lista de aristas**.

Después, qué pregunta hace el algoritmo

Se puede cambiar de representación

Pasar de lista de aristas a lista de adyacencia cuesta $\Theta(V + E)$, una sola pasada. Si el algoritmo va a hacer más trabajo que eso, conviene construir la representación que le sirve en lugar de forzarlo sobre la que llegó.

Leer un grafo de la entrada

El formato de siempre

La primera línea trae n y m; después vienen m líneas con los dos extremos de cada arista, numerados de 1 a n.

```
def leer_grafo(entrada):
    # Primera linea: n y m. Despues, m lineas con una arista cada una
    datos = entrada.read().split()
    n = int(datos[0])
    m = int(datos[1])
    G = []
    u = 0
    while u < n:
        G.append([])
        u = u + 1
    i = 0
    while i < m:
        u = int(datos[2 + 2 * i]) - 1
        v = int(datos[3 + 2 * i]) - 1
        G[u].append(v)
        G[v].append(u)
        i = i + 1
    return (n, m, G)
```

Leer un grafo de la entrada

Los dos detalles

Se lee todo de una vez y se parte por espacios, porque leer línea por línea es lento cuando hay cientos de miles de aristas. Y se resta 1 al leer, una sola vez.

Errores comunes

Los cuatro que más cuestan

- Olvidar la arista de vuelta en un grafo no dirigido. El programa corre, no falla, y contesta mal: el grafo quedó dirigido.
- Mezclar la numeración del enunciado con la de Python. Se resta 1 al leer y nunca más.
- Crear las listas con `[[]] * n`, que deja n referencias a la misma lista. Hay que crear una lista nueva por vértice.
- Usar matriz de adyacencia cuando n pasa de unos pocos miles. La memoria se acaba antes de que el algoritmo empiece.

Un vértice aislado no es un error

El 6 de G_1 tiene lista vacía y fila de ceros. Los algoritmos deben funcionar con él, y suele ser el caso de prueba que se olvida.

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

8 Cuál representación usar

9 Ejercicios

10 Referencias

Para practicar en papel

Sobre el grafo G_1

1. Escriba $N(1)$ y $N(6)$, y la secuencia de grado del grafo.
2. Verifique el apretón de manos contando las aristas.
3. Escriba G_1 en las tres representaciones y confirme que dan los mismos vecinos para el vértice 1.

Sobre las familias

1. ¿Existe un grafo simple con secuencia de grado $3, 3, 3, 3, 2$? ¿Y con $1, 2, 3, 4, 4$? ¿Y con $1, 2, 3, 4, 5$? Use el apretón de manos antes de intentar dibujarlos.
2. ¿Cuántos vértices tiene un grafo regular de grado 4 con 10 aristas?
3. Dibuje el complemento de C_5 y diga qué familia resulta.

Para enviar al juez

UVa 10928 — My Dear Neighbours

Enunciado:
`https://vjudge.net/problem/UVA-10928`
Envío:
`https://vjudge.net/problem/UVA-10928`

Cómo atacarlo

Cada línea de la entrada trae los vecinos de un lugar, y no dice cuántos son: hay que partir la línea y contar cuántos quedaron. El enunciado avisa que si P_1 tiene a P_2 como vecino no se sigue lo contrario, así que el grafo es dirigido y lo que se pide es el grado de salida. Se imprimen todos los lugares que empatan en el mínimo, ordenados. Los casos van separados por una línea en blanco.

Uno más, sobre la matriz de incidencia

UVa 11550 — Demanding Dilemma

Enunciado:
`https://vjudge.net/problem/UVA-11550`
Envío:
`https://vjudge.net/problem/UVA-11550`

Cómo atacarlo

Dan una matriz de $n \times m$ y preguntan si puede ser la matriz de incidencia de un grafo simple no dirigido. Las dos condiciones salen de la definición: cada columna representa una arista, así que debe tener exactamente dos unos, y como el grafo es simple no puede haber dos aristas entre el mismo par, es decir, dos columnas iguales.

Lo que sigue

Recorrer el grafo

Con el grafo ya guardado, la pregunta siguiente es cómo visitarlo: a qué vértices se llega desde uno dado, en qué orden y con qué costo. De ahí salen la búsqueda en profundidad y la búsqueda en amplitud, y sobre ellas se construye casi todo el resto del curso.

Para investigar

Busque en qué se diferencian recorrer un grafo con una pila y recorrerlo con una cola, y qué pasa si un vértice se puede alcanzar por dos caminos distintos.

Contenido

1 De las estructuras lineales a los grafos

2 Introducción a los grafos

3 Terminología

4 Familias de grafos simples

5 Topologías y subgrafos

6 Grafos complementarios

7 Representaciones

8 Cuál representación usar

9 Ejercicios

10 Referencias

Referencias


Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009. Apéndice B.4 (pp. 1168–1172) y Sección 22.1 (pp. 589–593).


Rosen, K. H. *Discrete Mathematics and Its Applications*, 7th ed. McGraw-Hill, 2012. Capítulo 10.


van Steen, M. *Graph Theory and Complex Networks: An Introduction*. 2010.


Halim, S., Halim, F., Effendy, S. *Competitive Programming 4*. Lulu, 2020. Sección 2.4.1.